

Тема 4 Проектирование реляционных баз данных с использованием нормализации

Общие положения

Проектирование базы данных является одной из наиболее сложных и ответственных задач, связанных с созданием и реализацией базы данных.

Фактически проект БД – это фундамент будущего программного средства.

В целом структурно основные задачи курсового проектирования отображены на рисунке 2.1



Рисунок 2.1 – Основные задачи курсового проектирования

Процесс проектирования БД представляет собой последовательность переходов от неформального словесного описания объектов предметной области к формализованному её описанию в терминах некоторой модели.

В общем случае можно выделить следующие этапы проектирования:

1. Системный анализ и словесное описание информационных объектов предметной области.

2. Проектирование инфологической модели – частично формализованное описание объектов предметной области в терминах некоторой семантической модели, например, в терминах ER-модели.

3. Дatalogическое (логическое проектирование) БД: описание БД в терминах принятой дatalogической модели данных.

4. Физическое проектирование БД: выбор эффективного размещения БД для обеспечения наиболее эффективной работы приложения

Учитывая то, что между 2-м и 3-м этапами нужно выбрать СУБД где будет реализовываться проект, то этапы проектирования БД можно представить структурно, как показано на рисунке 2.2.



Рисунок 2.2 – Этапы проектирования БД

Основой для анализа предметной области являются документы, позволяющие определить объекты и их свойства – что станет фундаментом проектируемой базы данных.

Общение с сотрудниками, для которых проектируется база данных, позволит получить сведения об ее особенностях, определить ограничения целостности, а также зависимости и связи между объектами предполагаемой базы данных.

Последовательность проектирования базы данных

Процесс проектирования базы данных включает в себя:

- определение задач, решаемых базой данных;
- сбор и анализ соответствующей информации предметной области, относящейся к решению задач проектируемой базы данных;
- описание особенностей предметной области, позволяющей определить зависимости и связи между ее объектами;
- определение групп пользователей и их задач и запросов к базе данных;

- выбор и обоснование аппаратной и программной платформы для реализации базы данных;
- создание – обобщенной и расширенной ER-модели базы данных;
- создание реляционных отношений;
- нормализация отношений до третьей нормальной формы;
- создание логической схемы базы данных до и после нормализации;
- определение типов и размеров данных атрибутов и ограничений целостности;
- создание физической структуры базы данных;
- создание таблиц базы данных и их заполнение средствами языка SQL в синтаксисе выбранной СУБД;
- определение прав доступа пользователей к объектам базы данных;
- создание (представлений, индексов, прав доступа и т.д.).

Анализ предметной области

На данном этапе делается подробное словесное и графическое описание объектов предметной области и реальных связей между ними.

Используя язык UML, желательно создавать различные виды диаграмм, позволяющих визуализировать описание предметной области.

Основными типами диаграмм являются:

- диаграмма вариантов использования;
- диаграмма взаимодействия
- диаграмма деятельности и иные диаграммы.

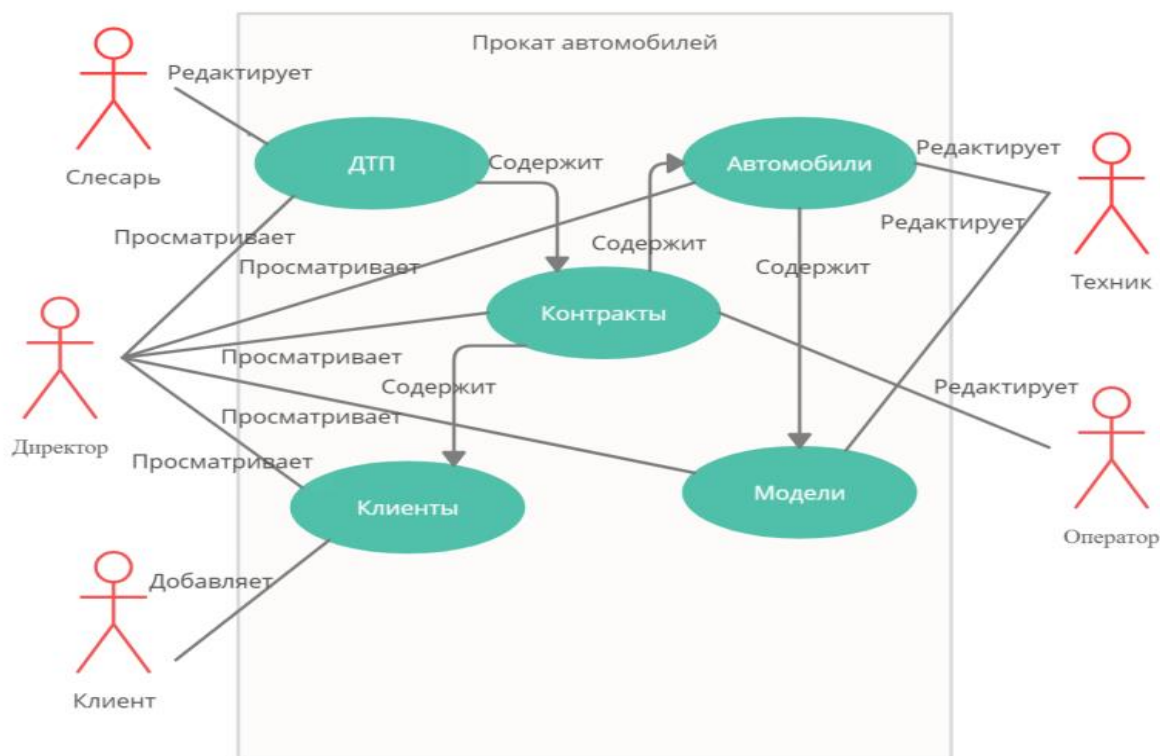


Рисунок 2.1 Пример диаграммы вариантов использования

Системный анализ завершается:

- подробным неформальным описанием объектов предметной области, которое требуется для решения конкретных задач базой данных;
- формулировкой конкретных задач, которые будут решаться с использованием данной базы данных с кратким описанием алгоритмов их решения;
- описанием выходных данных, которые должны генерироваться программным средством;
- описанием входных данных, которые служат основанием для заполнения базы данных.

Инфологическое проектирование

Данный этап проектирования называется концептуальным этапом проектирования базы данных, а ее результат – концептуальной моделью предметной области.

На данном этапе создается семантическая модель предметной области, проектируемой базы данных

Основными понятиями реляционного подхода к проектированию базы данных являются понятия сущности и атрибута сущности, как показано в таблице 2.1.

Таблица 2.1 – Понятия сущности и связи

Название	Определение
Сущность	это некоторый объект предметной области, который может существовать независимо. Сущность имеет экземпляры, отличающиеся друг от друга значениями атрибутов и допускающие однозначную идентификацию.
Атрибут	это свойство сущности. Атрибут, который уникальным образом идентифицирует экземпляры сущности, называется ключом.

Для визуализации процесса проектирования базы данных используется графический язык представления информации – язык UML.

Диаграммы на языке UML создаются с помощью CASE средств – набора инструментов и методов, позволяющих автоматизировать проектирование баз данных.

В настоящее время одним из средств формализованного представления предметной области на этом этапе, является модель Чена "сущность—связь", которая стала фактическим стандартом при инфологическом моделировании базы данных и общепринятым сокращенным названием – ER-модель.

При разработке ER-моделей необходимо получить следующую информацию о предметной области:

- 1 Список сущностей предметной области.
- 2 Список атрибутов сущностей.

3 Описание взаимосвязей между сущностями.
Элементы нотации Чена представлены в таблице 2.2.

Таблица 2.2 – Объекты нотации Чена

Вид элемента	Пояснение
Клиент	Каждый тип сущности в нотации Чена представлен в виде прямоугольника, содержащего имя сущности
Номер клиента в базе	Атрибуты сущностей отображаются в виде эллипсов, содержащих имя атрибута.

Эллипс соединяется с соответствующей сущностью линией.

Сущность должна определяться таким набором атрибутов, который позволял бы различать отдельные экземпляры сущностей, например, как показано на рис. 2.2.



Рисунок 2.2 – Пример описания атрибутов сущности

В таблице 2.3 описываются виды атрибутов.

Таблица 2.3 – Виды атрибутов

Виды атрибутов	Описание
<i>Идентифицирующие</i>	позволяют отличить один экземпляр сущности от другого.
<i>Описательные</i>	заключают в себе интересующие нас свойства сущности
<i>Составные</i>	является комбинацией нескольких элементов, возможно, принадлежащих разным типам данных (ФИО, адрес и др.)
<i>Простые</i>	атрибут имеет неделимое значение
<i>Однозначные</i>	имеет одно значение для каждого экземпляра сущности (например, дата рождения)
<i>Многозначные</i>	имеет много значений для каждого экземпляра сущности (например, номер телефона)
<i>Основные</i>	значение основного атрибута не зависит от других атрибутов
<i>Производные</i>	значение производного атрибута вычисляется на основе значений других атрибутов (например, возраст вычисляется на основе даты рождения и текущей даты)
<i>Обязательные</i>	должны быть указаны при размещении данных в базе данных
<i>Необязательные</i>	могут не указываться

Каждый экземпляр сущности должен отличаться от любого другого экземпляра той же сущности.

Сущность имеет имя, уникальное в пределах Ег-модели

На рисунке 2.3 показано, что между сущностями на Ег-модели могут быть установлены связи, отображающие, как сущности взаимодействуют между собой.

Связь – это осмысленная ассоциация между сущностями. Число сущностей, которое может быть ассоциировано через набор связей с другой сущностью, называют **степенью связи**. Связь между двумя сущностями называется бинарной, а связь между более чем с двумя сущностями – тернарной. На ER-диаграмме связь изображается ромбом.



Рисунок 2.3 – Ег-модель предметной области

Связь может существовать между двумя разными сущностями или между сущностью и ей же самой (рекурсивная связь).

Между двумя сущностями может быть задано сколько угодно связей с разными смысловыми нагрузками.

Логическое проектирование реляционной БД

На данном этапе разрабатывается логическая структура БД. Для реляционной модели существуют формальные правила, которые позволяют преобразовать инфологическую модель предметной области в виде ER-диаграммы в логическую схему базы данных. Кроме получения схемы базы данных в целом на этом этапе описываются реляционные отношения и осуществляется их нормализация.

Таким образом, главной задачей логического проектирования является представление выделенных на предыдущем этапе сведений в виде данных в форматах, поддерживаемых выбранной СУБД.

Этот этап более подробно рассмотрен в п.3 «Пример проектирования реляционной базы данных».

Важными понятиями реляционной алгебры на данном этапе являются:

Кортеж – это множество, которое содержит одно вхождение каждого атрибута, принадлежащего схеме отношения, т.е. – это набор именованных значений заданного типа.

Отношение – это множество кортежей, соответствующих одной схеме отношения (конечное множество атрибутов некоторого базового типа).

Простым представлением отношения является двумерная таблица, заголовком которой является схема отношения, а строками - кортежи отношения-экземпляра; в этом случае имена атрибутов именуют столбцы этой таблицы. Поэтому иногда говорят "столбец таблицы", имея в виду «атрибут отношения».

Понятия реляционного подхода отображены на рисунке 2.4.

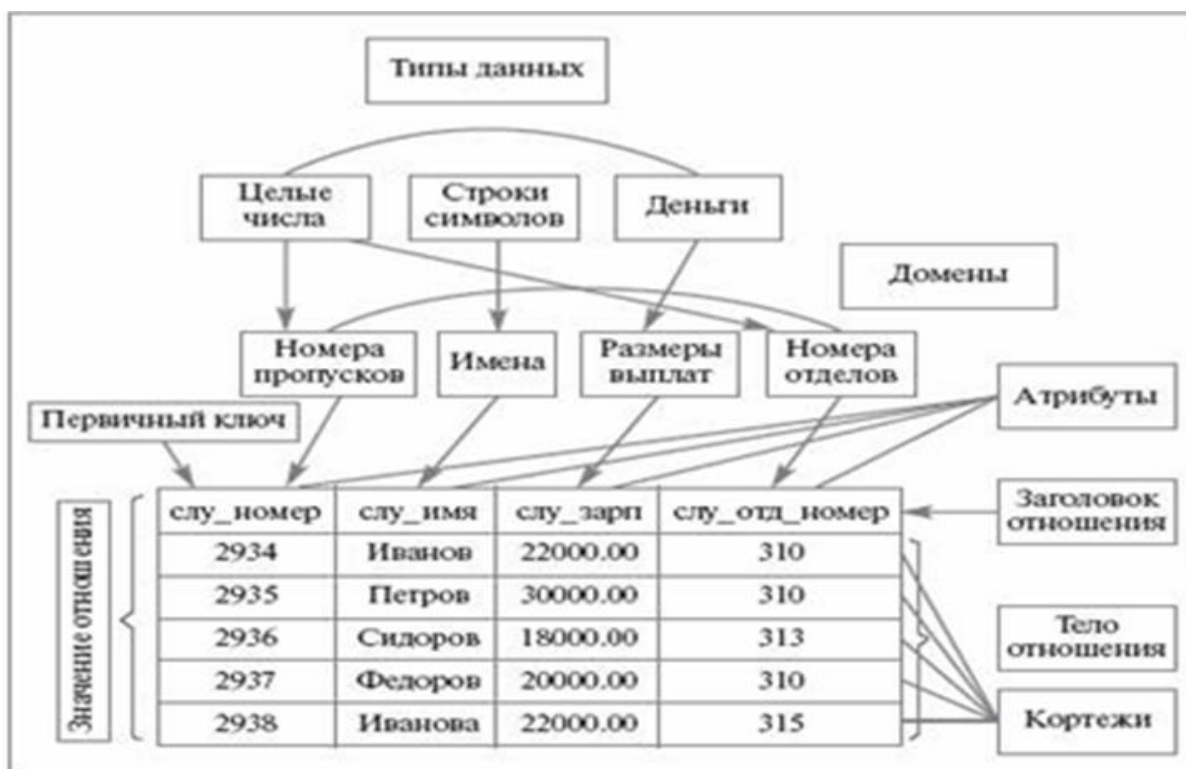


Рисунок 2.4 – Основные понятия реляционного подхода

Главными свойствами отношений, являются следующие свойства:

- 1 Отсутствие кортежей-дубликатов.
- 2 Отсутствие упорядоченности кортежей.
- 3.Отсутствие упорядоченности атрибутов.
- 4.Атомарность значений атрибутов.
- 5.В реляционных базах данных допускаются только нормализованные отношения.

Первичный ключ служит для идентификации записей и всегда имеет уникальное значение.

В отношении может быть назначен только один первичный ключ, у составного ключа атрибуты могут повторяться, но не все;

В первичном ключе не должно быть полей, которые, будучи удаленными из

него, не нарушат его уникальность;

Первичный ключ, как правило – числовое значение. Но он также может быть и любым другим типом данных, кроме атрибута типа комментариев и графического.

Первичный ключ может включать один либо несколько атрибутов, например, двух (трех, четырех...). Так как первичный ключ всегда уникален, а значит нужно, чтобы комбинация n-го количества атрибутов, в конкретной комбинации, была уникальна.

На логическом этапе проектирования БД важной характеристикой связи является тип связи (кардинальность)

Для связи указывается название, кардинальность (1:1, 1:n или m:n) и степень (бинарная, тернарная).

Каждая связь имеет два конца и одно или два наименования. Наименование обычно выражается в неопределенной глагольной форме: "иметь", "принадлежать" и т.п. Каждое из наименований относится к своему концу связи. Иногда наименования не пишутся ввиду их очевидности.

Краткое описание типов связи приведено в таблице 2.4.

Таблица 2.4 – Типы связей

Тип связи	Описание
один-к-одному	любому экземпляру сущности <u>A</u> соответствует только один экземпляр сущности B, и наоборот
один-ко-многим	любому экземпляру сущности <u>A</u> соответствует 0, 1 или несколько экземпляров сущности B, но любому экземпляру сущности B соответствует только один экземпляр сущности A
многие-к-одному	любому экземпляру сущности <u>A</u> соответствует только один экземпляр сущности B, но любому экземпляру сущности B соответствует 0, 1 или несколько экземпляров сущности A
многие-ко-многим	любому экземпляру сущности <u>A</u> соответствует 0, 1 или несколько экземпляров сущности B, и любому экземпляру сущности B соответствует 0, 1 или несколько экземпляров сущности A. Связь создается с помощью трех таблиц. Две таблицы – “источника” и одна соединительная таблица. Первичный ключ соединительной таблицы A_B – составной. Она состоит из двух полей, двух внешних ключей, которые ссылаются на первичные ключи таблиц A и B

Каждая связь может иметь одну из двух модальностей связи, изображенных на рисунке 2.5.

Связь может быть *обязательной*, если в данной связи должен участвовать каждый экземпляр сущности.

Связь может быть *необязательной*, если не каждый экземпляр сущности может участвовать в данной связи.

Модальность «может» означает, что экземпляр одной сущности может быть связан с одним или несколькими экземплярами другой сущности, а может быть и не связан ни с одним экземпляром.

Модальность «должен» означает, что экземпляр одной сущности обязан быть связан не менее чем с одним экземпляром другой сущности.

Может

Должен

Рисунок 2.5 – Модальности связей

Связь может иметь разную модальность с разных концов.

Описанный графический синтаксис позволяет однозначно читать диаграммы, пользуясь следующей схемой построения фраз:

<Каждый экземпляр СУЩНОСТИ А> <МОДАЛЬНОСТЬ СВЯЗИ>
<НАИМЕНОВАНИЕ СВЯЗИ> <ТИП СВЯЗИ> <экземпляр СУЩНОСТИ В>

На этапе логического проектирования, по завершению описания отношений базы данных, на основе ER-модели создается реляционная модель базы данных (схема данных), как показано на рисунке 2.6.

Инфологическую модель данных, построенную в виде ER-диаграммы, следует преобразовать в схему БД

Преобразование ER-диаграммы в схему БД выполняется путем сопоставления каждой сущности и каждой связи, имеющей атрибуты, в таблицы-отношения (таблицы БД)

Для этого необходимо выполнить следующие шаги:

1. Представить независимую сущность таблицей базы данных (базовой таблицей) и специфицировать первичный ключ этой таблицы.

2. Представить каждую ассоциацию (связь вида «многие-ко-многим» и т.д. между сущностями) как базовую таблицу.

3. Использовать в этой таблице внешние ключи для идентификации участников ассоциации и специфицировать ограничения, связанные с каждым из этих внешних ключей.

4. Представить каждую дочернюю таблицу как базовую таблицу с введением внешних ключей идентифицирующих родительскую сущность

Специфицировать ограничения на внешний ключ этой таблицы и её первичный ключ, что гарантирует уникальность в рамках описываемой сущности.

5. Представить каждое свойство как поле в базовой таблице, представляющей сущность, которая непосредственно описывается этим свойством

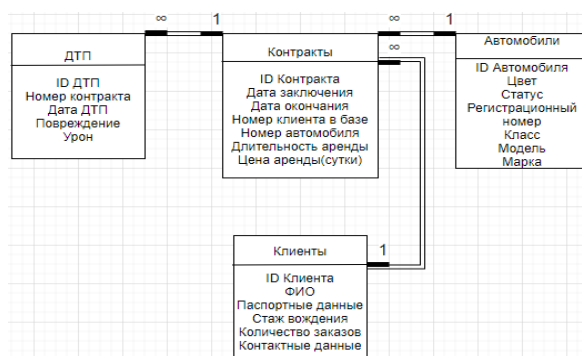


Рисунок 2.6 – Пример логической схемы данных до нормализации

Каждое реляционное отношение соответствует одной сущности предметной области и всем атрибутам этой сущности.

Проектирование схемы базы должно решать задачи исключения дублирования данных и упрощения процедур их обработки и модификации.

Для решения подобных проблем проводится **нормализация отношений**.

Нормализация – это процесс организации данных в базе данных, включающий создание таблиц и установление связей между ними в соответствии с правилами, которые обеспечивают защиту данных и делают базу данных более гибкой, устраняя избыточность и несогласованные зависимости.

Нормализация схемы отношения выполняется путём декомпозиции схемы отношения R и замены её совокупностью схем отношений A_i

Цель нормализации: исключить избыточное дублирование данных, которое является причиной аномалий, возникших при добавлении, редактировании и удалении кортежей (строк таблицы).

Аномалией называется такая ситуация в таблице БД, которая приводит к противоречию в БД либо существенно усложняет обработку БД.

Причиной является излишнее дублирование данных в таблице, которое вызывается наличием функциональных зависимостей от не ключевых атрибутов.

Существует несколько правил нормализации БД. Каждое правило называется «нормальной формой»

Каждое правило называется «нормальной формой»

Если выполняется первое правило, то говорят, что БД представлена в «первой нормальной форме»

Если выполняются три первых правила, считается, что БД представлена в «третьей нормальной форме»

Введём понятие простого и сложного атрибута:

Простой атрибут – это атрибут, значения которого атомарны (т.е. неделимы)

Сложный атрибут – атрибут представляет собой конкатенацию нескольких значений одного или разных доменов.

Проекция — отношение, полученное из заданного путём удаления и (или) перестановки некоторых атрибутов.

Отношение находится в 1НФ, если:

- все его атрибуты являются простыми
- все используемые домены должны содержать только скалярные значения;
- не должно быть повторений строк в таблице.

Другими словами, первая нормальная форма (1НФ):

- запрещает столбцы, содержащие одинаковую по смыслу информацию;
- запрещает столбцы, содержащие значения типа списка и т.п.);
- требует определить тот столбец или комбинацию столбцов, которые однозначно определяют каждую строку.

Таким образом, отношение находится в *первой нормальной форме (1НФ)* тогда и только тогда, когда ни одна из ее строк не содержит в любом своем поле более одного значения и ни одно из ее ключевых полей не пусто

Введём понятие функциональной зависимости. Пусть X и Y – атрибуты (группы атрибутов) некоторого отношения.

Говорят, что Y функционально зависит от X , если в любой момент времени каждому значению $X=x$ соответствует единственное значение $Y=y$.

При этом любому значению $Y=y$ может соответствовать несколько значений $X=(x_1, x_2, \dots)$.

Отношение находится во 2НФ, если оно приведено к 1НФ и каждый не ключевой атрибут функционально полно зависит от первичного ключа.

Для того чтобы привести отношение ко 2НФ, нужно:

1. построить его проекцию, исключив атрибуты, которые не находятся в функционально полной зависимости от составного ключа;
2. построить дополнительные проекции на часть составного ключа и атрибуты, функционально зависящие от этой части ключа.

Таким образом, отношение находится во *второй нормальной форме (2НФ)*, если она удовлетворяет определению 1НФ и все ее поля, не входящие в первичный ключ, связаны полной функциональной зависимостью с первичным ключом.

Рассмотрим понятие транзитивной зависимости

Пусть X, Y, Z – атрибуты некоторого отношения. При этом $X \rightarrow Y$ и $Y \rightarrow Z$, но обратное соответствие отсутствует, т.е. Z не зависит от Y или Y не зависит от X . Тогда говорят, что Z транзитивно зависит от X ($X \rightarrow \rightarrow Z$).

Отношение находится в 3НФ, если оно находится во 2НФ и в нем отсутствуют транзитивные зависимости.

Чтобы отношение находилась в 3НФ, необходимо, чтобы не ключевые столбцы в ней не зависели от других не ключевых столбцов, а зависели только от первичного ключа.

Отношение находится в 3НФ, когда находится во 2НФ и каждый не ключевой атрибут нетранзитивно зависит от первичного ключа.

Проще говоря, второе правило требует выносить все не ключевые поля, содержимое которых может относиться к нескольким записям таблицы в отдельные таблицы.

После нормализации отношений, необходимо создать итоговую схему базы данных, как показано на рисунке 2.7.

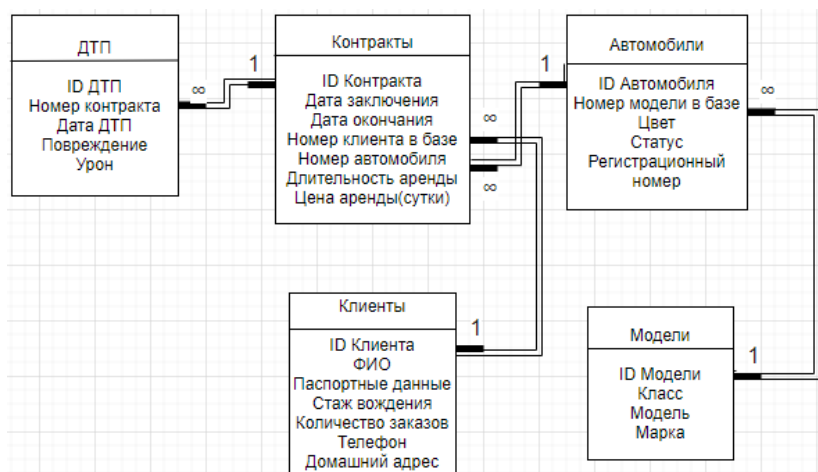


Рисунок 2.7 – Пример логической схемы данных после нормализации

Определение требований к операционной обстановке

На этом этапе осуществляется оценка требований к ресурсам компьютерной системы, необходимым для функционирования базы данных, определение типа и конфигурации конкретного ПК, выбор типа и версии операционной системы. Объём вычислительных ресурсов зависит от предполагаемого объёма проектируемой базы данных и от интенсивности их использования.

Если база данных будет работать в многопользовательском режиме, то требуется подключение её к сети и наличие соответствующей многозадачной операционной системы [1]. Пример описания требований к операционной обстановке сделан в п.3.2

Выбор СУБД и других программных средств

Выбор СУБД осуществляется на основании таких критериев, как тип модели данных и её адекватность потребностям рассматриваемой предметной области; характеристики производительности; набор функциональных возможностей; удобство и надёжность СУБД в эксплуатации; стоимость СУБД и дополнительного программного обеспечения [1].

Физическое проектирование БД

Физическая структура базы данных определяет тип и свойства данных, которые будут записаны в память компьютера.

На данном этапе необходимо подробно описать структуры таблиц данных с помощью типов данных, свойств полей, поддерживаемых выбранной СУБД, обосновать выбор индексных полей.

Пример физической структуры базы данных показан на рисунке 2.8.

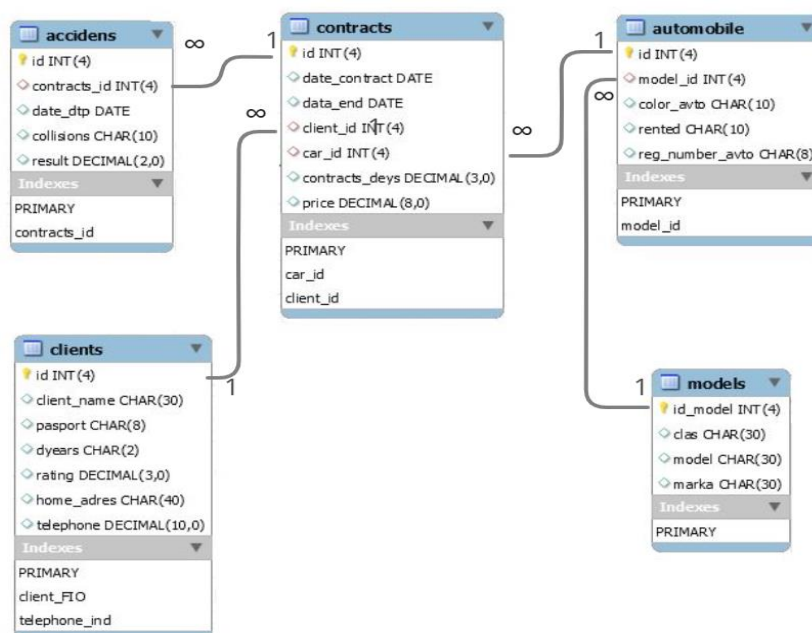


Рисунок 2.8 – Пример физической структуры базы данных

Правила перехода к физической модели следующие:
каждое отношение превращается в файл базы данных,
каждый столбец – в поле файла,
каждая строка – в запись файла.

Этап физического проектирования БД включает в себя определение состава файлов и их заполнение исходными данными в соответствии с ограничениями, допущениями и особенностями предметной области.

Физическая структура базы данных, с одной стороны, должна адекватно отражать ее логическую структуру, а с другой стороны, должна обеспечивать эффективное размещение данных и быстрый доступ к ним. Результаты этого этапа документируются в форме схемы хранения на языке определения данных DDL языка SQL в выбранной СУБД. Принятые на этом этапе решения оказывают значительное влияние на производительность программного средства.

Одной из важных составляющих в проектировании базы данных является разработка средств защиты БД.

Защита данных имеет два аспекта: защита от сбоев и защита от несанкционированного доступа. Для защиты от сбоев на этапе физического проектирования разрабатывается стратегия резервного копирования. Для защиты от несанкционированного доступа каждому пользователю доступ к данным предоставляется только в соответствии с его правами доступа, набор которых также является составной частью проекта базы данных.

Таким образом, задача физического проектирования — выбор способа хранения данных на физических носителях и методов доступа к ним с использованием возможностей, предоставляемых СУБД.